

J.A.R.V.I.S — ET AI Hackathon 2026 Sprint Plan

Hackathon: ET AI Hackathon 2026 — Problem #8 (Industrial Knowledge Intelligence) **Repo (sanitized):** <https://github.com/Aayushashsahu/J.A.R.V.I.S.git> **Working folder:** `C:\J.A.R.V.I.S (hackthone)\` **Sprint window:** 30 hours total (planned for a 2-day burst; aim to submit at Hour 30 for early-bird bonus) **Goal:** Working Industrial Knowledge Intelligence platform that demos end-to-end on real OISD / Factory Act / BIS / PESO documents with citation-backed answers, agent orchestration, and a graph-of-thought knowledge view.

1. Mission (in one sentence)

Ship a working J.A.R.V.I.S demo that ingests hazardous-process documents, answers inspector-style questions with citations, runs multi-agent research, and visualizes the knowledge graph — and get it submitted with a clean README, deployed link, and architecture diagram.

2. The Four Roles

Person	Role	Primary Output
Aayush	Architect, Integrator, Pitch Owner	Repo health, integration, submitting, demo rehearsal
Navansh	RAG + Vector Lead	<code>/api/chat</code> real-DB plumbing, hybrid search, embeddings
Yash	Agent + Reasoning Lead	<code>/api/agent/orchestrate</code> , multi-agent flow, KG build
Nirupam Pal	Frontend + UX Lead	Dashboard, query UI, graph viz, citations panel

The original JARVIS code (Aayush's personal work) is preserved separately at `C:\J.A.R.V.I.S.zip` . Any rewrite happens in `C:\J.A.R.V.I.S (hackthone)\` only.

3. Aayush — Architect, Integrator, Pitch Owner

Mission: *Protect the existing codebase, integrate teammates' modules, ensure the demo works, and deliver submission.*

This is the role definition you asked for. Below is the hour-by-hour table you specified, locked.

Aayush's hour-by-hour deliverables

Hour	What must be done	Visible output
0–1	Create branches	<code>main</code> protected; <code>feat/navansh-rag</code> , <code>feat/yash-agents</code> , <code>feat/nirupam-ui</code> , <code>feat/aayush-infra</code> all pushed; branch protection set (no force-push to <code>main</code>).
1–2	Write <code>CONTRACTS.md</code> , <code>README.md</code> , <code>DEMO_SCRIPT.md</code>	All three committed to <code>main</code> . <code>CONTRACTS.md</code> freezes the public API shapes for the next 26 hours.
2–4	Smoke-test the stack, wire first endpoints	<code>docker compose up</code> brings up Postgres + Qdrant + Redis + backend + frontend with zero errors; <code>/api/chat</code> returns at least one canned response; <code>/api/agent/orchestrate</code> returns 200 on a stub.
4–8	Standby, unblock, review contract compliance	Sit in the group DM. PR-by-PR review only — no code writing unless Navansh/Yash/Nirupam are blocked on infra. Pin a checklist of frozen contracts.
8–14	First end-to-end integration	Three teammates' branches merged behind feature flags; <code>/api/chat</code> + <code>/api/agent/orchestrate</code> + dashboard query all run on the <i>same</i> dataset.
14–18	Architecture diagram + docs	Draw.io → PNG of full system (Ingest → Embed → Qdrant → Retriever → LLM → Frontend); embed in README. C4-style

Hour	What must be done	Visible output
		diagram for the architecture section.
18–22	Integration testing, merge branches	All feature flags off; one final integration push to <code>main</code> ; smoke-test all four primary demo flows in production config.
22–24	Full demo rehearsal	Walk through <code>DEMO_SCRIPT.md</code> end-to-end twice. Time it. Identify the failure point. Fix immediately.
24–28	Final polish (README, arch diagram, screenshots)	Polished README with logo, demo GIF, architecture, install, env table, ethics note, team credits. Five screenshots.
28–30	Final rehearsal + submit	One last dry run. Submit the repository and deployment link. Hand off the next-sprint task list to Navansh (so Aayush is free post-submission).

Aayush owns (assigned into everyone's path)

- Repo health, `.env.example` , secrets, `.gitignore` .
- `docker-compose.yml` and the `gbrain` service config.
- `gbrain init --pglite --no-embedding` initialization step (documented).
- `feedback/kg-builder` glue code (Yash writes the agent; Aayush wires the NetworkX store into the backend startup).
- Submission ZIP and the public GitHub link.
- The 60-second pitch script (3 bullets + 1 close).

Aayush's pre-flight checks (Hour 0)

1. Confirm the working folder is `C:\J.A.R.V.I.S (hackthone)\` — *not* the personal JARVIS copy.
2. `git status` is clean. Origin points to `https://github.com/Aayushashsahu/J.A.R.V.I.S.git` .
3. `.env.example` has *no* live keys — placeholders only.
4. Docker Desktop is running and has at least 8 GB RAM allocated.

4. Navansh — RAG + Vector Lead

Hour	Deliverable
0–1	Read CONTRACTS.md; comment any blockers in group chat. Clone repo locally, set up <code>.env</code> .
2–4	Pull existing <code>app/services/retriever.py</code> and <code>app/services/embedding_service.py</code> into shape. Confirm Qdrant collection schema: 768-dim vectors, payload { <code>source</code> , <code>page</code> , <code>clause_id</code> , <code>text</code> }.
4–8	Hybrid search: BM25 over Postgres <code>documents</code> table + cosine over Qdrant. Reciprocal rank fusion. Must add <code>POST /api/ingest/upload</code> to accept a folder of OISD PDF docs.
8–14	Wire <code>/api/chat</code> to use the hybrid retriever + the response format promised in CONTRACTS.md (answer, citations[], chunk_ids[]). Stream tokens.
14–18	Cache embeddings in Postgres with a content-hash key. Add <code>/api/ingest/status/{job_id}</code> polling endpoint.
18–22	Submit a clean PR to <code>feat/navansh-rag</code> . Run the smoke tests from <code>tests/integration/test_chat.py</code> .
22–28	Standby for question / answer hardening. No new features — only edge-case catches.
28–30	Rehearse five demo questions with Aayush. Take one answer for the README screenshot.

Critical handoff: the `/api/ingest/upload` endpoint must accept any file path the frontend can supply; the response shape is fixed in CONTRACTS.md as `{job_id, accepted, total}`.

5. Yash — Agent + Reasoning Lead

Hour	Deliverable
0–1	Read CONTRACTS.md. Branch <code>feat/yash-agents</code> already exists. Pick a single-agent framework (LangGraph) and pin the version in <code>requirements.txt</code> .
2–4	Stub <code>/api/agent/orchestrate</code> returns canned JSON for the contract shape.

Hour	Deliverable
4–8	Implement a <i>single</i> -agent pipeline: question → planner → retriever (calls Navansh's RAG) → verifier → formatter. Use NVIDIA NIM (Llama 3.1 70B) as primary LLM; Gemini as fallback if NV key missing.
8–14	Multi-agent: add a "researcher" sub-agent that fans out 3 sub-questions and synthesises. Persist agent runs to the <code>agent_runs</code> table with full trace JSON.
14–18	Build the knowledge graph: NetworkX in-process, but expose <code>/api/kg/neighbors/{entity}</code> returning <code>{source, target, weight}[]</code> .
18–22	Frontend integration: <code>/api/agent/stream</code> — server-sent events for live token streaming.
22–28	Hardening: input validation (LangGraph state guard), timeout 60s on every agent run, cost-aware model selection.
28–30	Rehearse the multi-agent demo. README "How the agent thinks" section.

Critical handoff: the `/api/agent/orchestrate` request/response JSON is fixed in `CONTRACTS.md`. Do not change fields mid-sprint without a call to Aayush.

6. Nirupam Pal — Frontend + UX Lead

Hour	Deliverable
0–1	Read <code>CONTRACTS.md</code> . Check out <code>feat/nirupam-ui</code> . Run <code>npm install</code> and confirm dev server boots on <code>:3000</code> .
2–4	Skeleton dashboard: query box (left), answer pane with citations (right), file-upload panel (bottom). No styling perfection yet — just layout.
4–8	Wire <code><QueryBox /></code> to <code>POST /api/chat</code> . Render citations as clickable chips. Each chip scrolls the source-document viewer.
8–14	Add <code>/admin/ingest</code> page: drop-zone for PDFs, live progress bar polling <code>/api/ingest/status/{job_id}</code> .

Hour	Deliverable
14–18	Add knowledge-graph viewer: a small canvas with circles (entities) and lines (relations). Click a node → cite one source. Use <code>react-force-graph-2d</code> or a lighter lib (no heavy D3).
18–22	"Reasoning trace" panel: shows the live agent SSE stream — planner → retriever → verifier → final.
22–28	Polish: dark mode, mobile-friendly, keyboard shortcut for "ask". Add a "Reset demo" button. Bump Lighthouse score.
28–30	Screenshot all five flows. Embed in README. Rehearse the demo with Aayush.

Critical handoff: the frontend never holds secrets. NVIDIA key, Gemini key, JWT, anything sensitive — *only* in the backend env. Nirupam must pull these from `/api/config` (debug-only endpoint) if needed for a feature flag.

7. CONTRACTS.md (locked — do not change without Aayush's approval)

```
// POST /api/chat
// Request:
{
  "workspace_id": "demo",
  "question": "What is the minimum clearance for hydrocarbon storage under OISD-118?",
  "filters": { "clause": null, "source": null },
  "top_k": 6
}

// Response (streamed via NDJSON, one line per event):
{"type":"token", "delta":"..."}
{"type":"citation", "chunk_id":"oisd118_p12_c42", "source":"OISD-118.pdf", "page":12, "clause":"3.4"}
{"type":"final", "answer":"...", "citations":[...], "confidence":0.83}
```

```
// POST /api/agent/orchestrate
// Request:
{ "workspace_id":"demo", "goal":"Compare OISD-118 vs PESO for LPG storage", "max_steps":6 }
// Response (SSE):
```

```
{"event":"trace", "step":1, "node":"planner", "content":"..."}
{"event":"trace", "step":2, "node":"retriever", "sources":3}
{"event":"final", "answer":"...", "sources":[...], "agent_run_id":"..."}
```

```
// POST /api/ingest/upload
// Request: multipart/form-data, field name "files"
// Response: { "job_id":"...", "accepted":3, "total":3 }
// Subsequent: GET /api/ingest/status/{job_id} → { "state":"processing|done|error", "progress":0.42,
```

```
// GET /api/kg/neighbors/{entity}
// Response: { "entity":"OISD-118", "neighbors":[{"source":"PESO","target":"OISD-118","weight":0.81}]}
```

8. Communication rules (read these or lose time)

- **One group DM only.** No separate chats. Pin: [CONTRACTS.md](#) link, [sprint table](#) link, [demo script](#) link.
 - **Every PR needs a one-line summary on first line:** `[contract] ...` , `[secrets] ...` , `[demo] ...` — SO Aayush can review by tag.
 - **No midnight surprises.** Any breaking change to a contract needs a heads-up **30 min before** the merge.
 - **Standups:** every 4 hours, top of the hour. 3 lines max: *what I shipped, what's blocking me, what's next.*
 - **Blockers go to Aayush first**, not the group, when they touch `infra/secrets/submission`.
-

9. Demo script (the 60 seconds that matter)

1. **0–10s:** Drag three industrial PDFs (OISD-118, Factory Act 1948, PESO rules) onto the dashboard. Progress bar fills.
2. **10–25s:** Type: *"What is the minimum setback for LPG bulk storage above 1000 MT?"* → answer streams with citations; click the citation chip → source page jumps.

3. **25–45s:** Switch to **Agent mode**: "*Compare OISD-118 and PESO for fire-water reservation.*"
Watch the trace panel: planner → retriever → verifier → final.
4. **45–55s:** Click an entity (**PESO**) on the knowledge-graph view. Lines light up.
5. **55–60s:** One sentence close: "*Same citations, every time. Auditable by design.*"

10. Risk register (the things that *will* go wrong)

Risk	Mitigation
NVIDIA API key absent / rate-limit	Fallback path to Gemini already coded in <code>llm_provider.py</code> . Env var drives selection.
Qdrant memory pressure with 50+ PDFs	Disable in-memory ANN index if RAM < 6 GB; use quantised vectors.
NetworkX graph blows past 10k nodes	Cap <code>max_entities</code> per workspace. Default 2000.
SSE drops in Safari	Frontend reverts to polling <code>/api/agent/status/{id}</code> after 5s of no event.
GitHub Actions docker build OOM during submission	Pin base image to <code>python:3.11-slim</code> , multi-stage build.
Teammate offline for 4+ hours	Aayush picks up the smallest deliverable from that lane; pair-program recovery in 30 minutes.

11. Submission checklist (Hour 28–30)

- [] `README.md` is clean: problem statement, demo GIF, architecture diagram, install steps, env table, ethics note, team credits.
- [] `CONTRACTS.md` present and unchanged from this document.
- [] `LICENSE` (MIT or Apache-2.0).
- [] `.env.example` has zero live secrets.
- [] `.gitignore` excludes `vault/` , `gbrain_temp/` , `node_modules/` , `**/.git/` , `uploads/` .

- [] Repository is public: <https://github.com/Aayushashsahu/J.A.R.V.I.S> .
- [] Deployment URL is live (Render / Railway free tier) and the link works.
- [] Demo video (≤ 3 minutes) uploaded to the hackathon portal.
- [] Five screenshots embedded in README (Upload, Chat+Cite, Agent Trace, Knowledge Graph, Settings/Auth).
- [] Pitch script rehearsed twice.

12. Hand-off after submission

Aayush is "free" after Hour 30. The following tasks go to Navansh so the demo and the next-sprint prep don't sit on Aayush:

1. Open the GitHub issues list: `[demo]` tag for any tester feedback.
2. Triage the next week's worth of P0/P1 issues.
3. Coordinate with Nirupam on a follow-up UX pass on the agent trace panel.
4. Schedule the next sprint review with all four teammates.

If you have nothing else worth doing today, ping the team that submission button has been clicked.

13. Copy-paste messages for teammates

Paste as your first message in the group DM when the sprint clock starts:

→ **Navansh**

Hey — welcome to the team. J.A.R.V.I.S sprint is starting now. Read this first: `CONTRACTS.md` at <https://github.com/Aayushashsahu/J.A.R.V.I.S/blob/main/CONTRACTS.md> (I'll commit it in 30 min). You're RAG lead. Day 1 (0–8h):

1. Pull `feat/navansh-rag` from origin. Add `POST /api/ingest/upload` .
2. Implement hybrid search (BM25 + cosine) and RRF.
3. Wire `/api/chat` against the contract shape.

Open questions: ping me (Aayush) on infra/secrets issues only. Use the group chat for everything else. Tag your PRs `[contract]` if they touch `CONTRACTS.md`, `[secrets]` if they touch `env`, `[demo]` if it's user-facing. Standup every 4 hours, top of the hour. Profile pic is set so @ mention me with **architect** to escalate.

→ Yash

Hey — welcome. J.A.R.V.I.S sprint is starting now. Read `CONTRACTS.md` first (linked above). You're agent/reasoning lead. Day 1 (0–8h):

1. Pull `feat/yash-agents` . Lock framework to LangGraph; pin in `requirements.txt` .
2. Stub `/api/agent/orchestrate` to the contract shape today.
3. Single-agent flow first (planner → retriever → verifier → formatter).

LLM order: NVIDIA NIM (Llama 3.1 70B) **primary**, Gemini **fallback** — both are wired through `llm_provider.py` . No new secret additions. Tag PRs `[contract]` for shape changes. Standup every 4h. Ping me **architect** for infra/secrets/submission issues only.

→ Nirupam Pal

Hey — welcome. J.A.R.V.I.S sprint is starting now. Read `CONTRACTS.md` first (linked above). You're frontend lead. Day 1 (0–8h):

1. Pull `feat/nirupam-ui` . Run `npm install` . Confirm Next.js dev server on `:3000`.
2. Skeleton dashboard: query + answer + citations + upload tiles.
3. Wire `<QueryBox />` to `POST /api/chat` .

Frontend never holds secrets. If you need any flag value for a feature, expose it via `/api/config` (debug endpoint) and I'll add it. Tag PRs `[demo]` for user-facing changes. Standup every 4h. Ping me **architect** for infra/secrets/submission issues only.

14. The two things Aayush will *not* write

1. **No live secrets in the repo at submission.** Final scan: `git grep -nE '(nvapi-|sk-[a-zA-Z]_|AIza[A-Za-z0-9_-]{20,}|"eyJ)"' || echo "clean"`.
2. **No console.log statements at submit time.** Frontend, backend, scripts.

Both are enforced via a CI step Aayush drops in at Hour 22.

15. Definition of Done (every PR)

- [] Branch name starts with `feat/` , `fix/` , Or `docs/` .
- [] PR has a one-line summary line that begins with `[contract]` , `[secrets]` , `[demo]` , Or `[docs]` .
- [] At least one reviewer (`@Aayushashsahu` for `[contract]` , `[secrets]` , `[demo]`).
- [] `git grep` for live secrets is clean.
- [] All four demo flows (Upload → Chat → Agent → Graph) still pass smoke test.

If all five boxes are checked, Aayush merges in under 5 minutes.

End of plan. Good luck — and remember: 2 days, 4 people, 30 hours. Submit early; polish after.